Think Data Structures

Algorithms and Information Retrieval in Java

Version 1.1.0

Think Data Structures

Algorithms and Information Retrieval in Java

Version 1.1.0

Allen B. Downey

Green Tea Press

Needham, Massachusetts

Copyright © 2016 Allen B. Downey.

Green Tea Press
9 Washburn Ave
Needham, MA 02492

Permission is granted to copy, distribute, and/or modify this work under the terms of the Creative Commons Attribution-NonCommercial-ShareAlike 3.0 Unported License, which is available at https://greenteapress.com/dast/cc30.

The original form of this book is LaTeX source code. Compiling this code has the effect of generating a device-independent representation of the book, which can be converted to other formats and printed.

The LaTeX source for this book is available from https://greenteapress.com/dast/repo.

Contents

Preface **xi**

0.1 Prerequisites xii

1 Interfaces **1**

1.1 Why are there two kinds of `List`? 2

1.2 Interfaces in Java 2

1.3 The List interface 3

1.4 Exercise 1 5

2 Analysis of Algorithms **9**

2.1 Selection sort 10

2.2 Big O notation 13

2.3 Exercise 2 14

3 ArrayList **19**

3.1 Classifying MyArrayList methods 19

3.2 Classifying `add` 21

3.3 Problem Size 24

3.4 Linked Data Structures 24

3.5 Exercise 3 27

3.6 A note on garbage collection 30

4 LinkedList 31

4.1 Classifying `MyLinkedList` methods 31

4.2 Comparing `MyArrayList` and `MyLinkedList` 34

4.3 Profiling 35

4.4 Interpreting results 37

4.5 Exercise 4 39

5 Doubly-linked list 41

5.1 Performance profiling results 41

5.2 Profiling `LinkedList` methods 43

5.3 Adding to the end of a `LinkedList` 44

5.4 Doubly-linked list 47

5.5 Choosing a Structure 48

6 Tree traversal 51

6.1 Search engines 51

6.2 Parsing HTML 52

6.3 Using jsoup 54

6.4 Iterating through the DOM 56

6.5 Depth-first search 57

6.6 Stacks in Java 58

6.7 Iterative DFS 59

7 Getting to Philosophy 63

7.1 Getting started 63

7.2 Iterables and Iterators 64

7.3 `WikiFetcher` 66
7.4 Exercise 5 68

8 Indexer **71**
8.1 Data structure selection 71
8.2 TermCounter 73
8.3 Exercise 6 76

9 The Map interface **81**
9.1 Implementing `MyLinearMap` 81
9.2 Exercise 7 82
9.3 Analyzing `MyLinearMap` 83

10 Hashing **87**
10.1 Hashing 87
10.2 How does hashing work? 89
10.3 Hashing and mutation 91
10.4 Exercise 8 93

11 HashMap **95**
11.1 Exercise 9 95
11.2 Analyzing `MyHashMap` 97
11.3 The tradeoffs 98
11.4 Profiling `MyHashMap` 99
11.5 Fixing `MyHashMap` 100
11.6 UML class diagrams 102

12 TreeMap **105**

12.1 What's wrong with hashing? . . . 105
12.2 Binary search tree . . . 106
12.3 Exercise 10 . . . 108
12.4 Implementing a TreeMap . . . 110

13 Binary search tree **113**

13.1 A simple `MyTreeMap` . . . 113
13.2 Searching for values . . . 114
13.3 Implementing `put` . . . 116
13.4 In-order traversal . . . 118
13.5 The logarithmic methods . . . 119
13.6 Self-balancing trees . . . 121
13.7 One more exercise . . . 122

14 Persistence **123**

14.1 Redis . . . 124
14.2 Redis clients and servers . . . 125
14.3 Making a Redis-backed index . . . 126
14.4 Redis data types . . . 128
14.5 Exercise 11 . . . 130
14.6 More suggestions if you want them . . . 132
14.7 A few design hints . . . 133

15 Crawling Wikipedia **135**

15.1 The Redis-backed indexer . . . 135
15.2 Analysis of lookup . . . 138
15.3 Analysis of indexing . . . 139

15.4 Graph traversal 140
15.5 Exercise 12 141

16 Boolean search **145**

16.1 Crawler solution 145
16.2 Information retrieval 148
16.3 Boolean search 148
16.4 Exercise 13 149
16.5 `Comparable` and `Comparator` 152
16.6 Extensions 154

17 Sorting **157**

17.1 Insertion sort 158
17.2 Exercise 14 160
17.3 Analysis of merge sort 162
17.4 Radix sort 164
17.5 Heap sort 165
17.6 Bounded heap 167
17.7 Space complexity 168

Index **171**